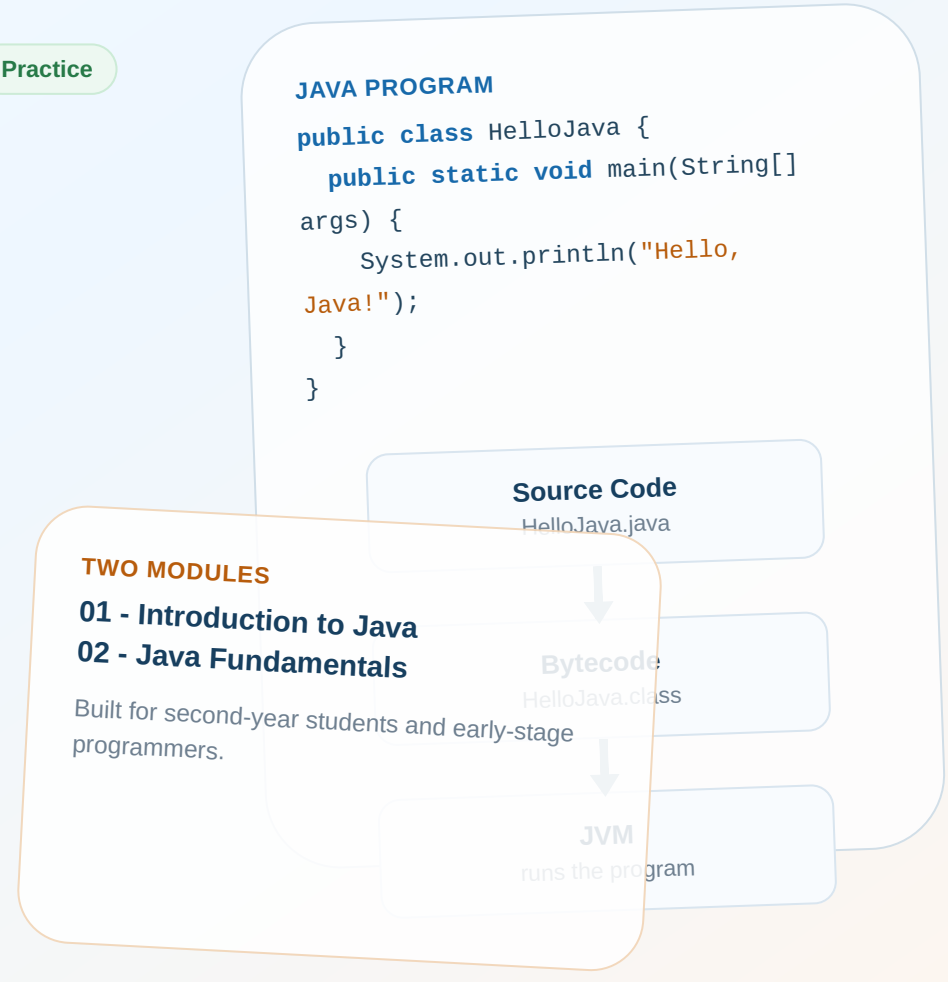
JAVA BEGINNER

Java Fundamentals Made Visual

Modules 1 & 2 - Introduction to Java + Java Fundamentals

A student-friendly learning guide designed to help you understand Java by scanning the idea, seeing the visual, reading the code, and then trying it yourself.

Learn Visualize Code Practice



START HERE

How to use this guide

Do not try to memorize every definition. Use each page in this order: understand the idea, inspect the visual, read the example, then test yourself.

1 - UNDERSTAND

Read the concept

Focus on what the term means and why Java needs it.

2 - VISUALIZE

Follow the diagram

See how the parts connect before learning syntax.

3 - PRACTICE

Run the code

Type examples yourself and change one thing at a time.

Contents

STUDY HABIT

Type the code instead of copying it. Small typing mistakes are useful because fixing them teaches you how Java syntax actually behaves.

MODULE 1

Introduction to Java

Before writing programs, build the mental model: what Java is, why it became popular, and how your source code becomes a running program.

AFTER THIS MODULE, YOU SHOULD BE ABLE TO

- Explain programming and the role of a programming language.
- Describe Java's main features, uses and editions.
- Differentiate JDK, JRE and JVM.
- Explain bytecode and the Java compilation process.
- Install Java, write a first program and identify its structure.

MENTAL MODEL

You write instructions.

```
Human idea
   |
   v
Java source code
   |
   v
Compiler + JVM
   |
   v
Running program
```

MODULE 1 - FOUNDATIONS

What is programming?

Programming is the process of giving a computer a precise set of instructions so it can solve a problem or perform a task.

SIMPLE EXPLANATION

Think: problem -> steps -> code

A computer does not guess what you mean. You break a problem into logical steps and express those steps using a programming language.

REAL-WORLD ANALOGY

A recipe is like a program

Ingredients are data. Recipe steps are instructions. The prepared dish is the output. If a step is missing or wrong, the result changes.

WHY THIS MATTERS

Learning syntax is only one part of programming. The more important skill is learning to convert a problem into clear, ordered steps.

What is Java?

Java is a **high-level, class-based, object-oriented programming language** designed to be readable, portable and suitable for many kinds of software.

- **High-level** - closer to human-readable code
- **Class-based** - programs are organized around classes
- **Portable** - bytecode can run on different JVM implementations

Quick Check: If a computer cannot understand human intention directly, what must a programmer provide instead?

MODULE 1 - CONTEXT

Java: history, features and applications

A short history

Java began at Sun Microsystems in the early 1990s. The language was initially called **Oak** and was later renamed Java. Java 1.0 was publicly released in 1996.

WHY JAVA STOOD OUT

The key idea was portability: compile source code into an intermediate format called **bytecode**, then run that bytecode using a Java Virtual Machine on the target platform.

Major features

- **Platform independent:** Java bytecode can run wherever a compatible JVM exists.
- **Object-oriented:** Java supports classes, objects, inheritance, polymorphism, encapsulation and abstraction.
- **Strongly typed:** variables and expressions follow defined data types.
- **Automatic memory management:** Java uses garbage collection to reclaim unreachable objects.
- **Rich ecosystem:** Java has extensive libraries, tools and frameworks.

WHERE JAVA IS USED

Backend systems

Web APIs, services and large server-side applications.

Enterprise apps

Business systems used by banks, companies and institutions.

Android ecosystem

Java remains part of Android development, alongside Kotlin.

Tools & platforms

Build tools, IDE features, data platforms and developer utilities.

REMEMBER THIS

"Write once, run anywhere" is a popular way to describe Java portability. In practice, the target environment still needs a compatible Java runtime and application dependencies.

INTERVIEW CONNECTION

Beginner interviews often ask why Java is platform independent. The short answer: **Java source is compiled to bytecode, and the JVM executes that bytecode on each supported platform.**

MODULE 1 - JAVA ECOSYSTEM

Java editions: SE, EE and ME

The editions describe different areas of the Java platform. For beginners, Java SE is the foundation you learn first.

JAVA SE

Standard Edition

Core Java platform used to learn and build general-purpose Java applications.

- Language basics
- Collections
- I/O
- Concurrency

JAVA EE

Enterprise Edition

Historically used for enterprise and server-side Java specifications.

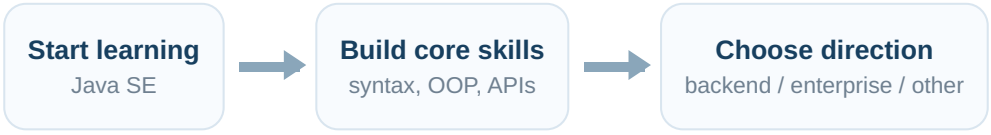
Modern name: Java EE was transferred to the Eclipse Foundation and continues as **Jakarta EE**.

JAVA ME

Micro Edition

A Java platform designed for resource-constrained and embedded devices.

- Embedded
- Small devices



DO NOT CONFUSE

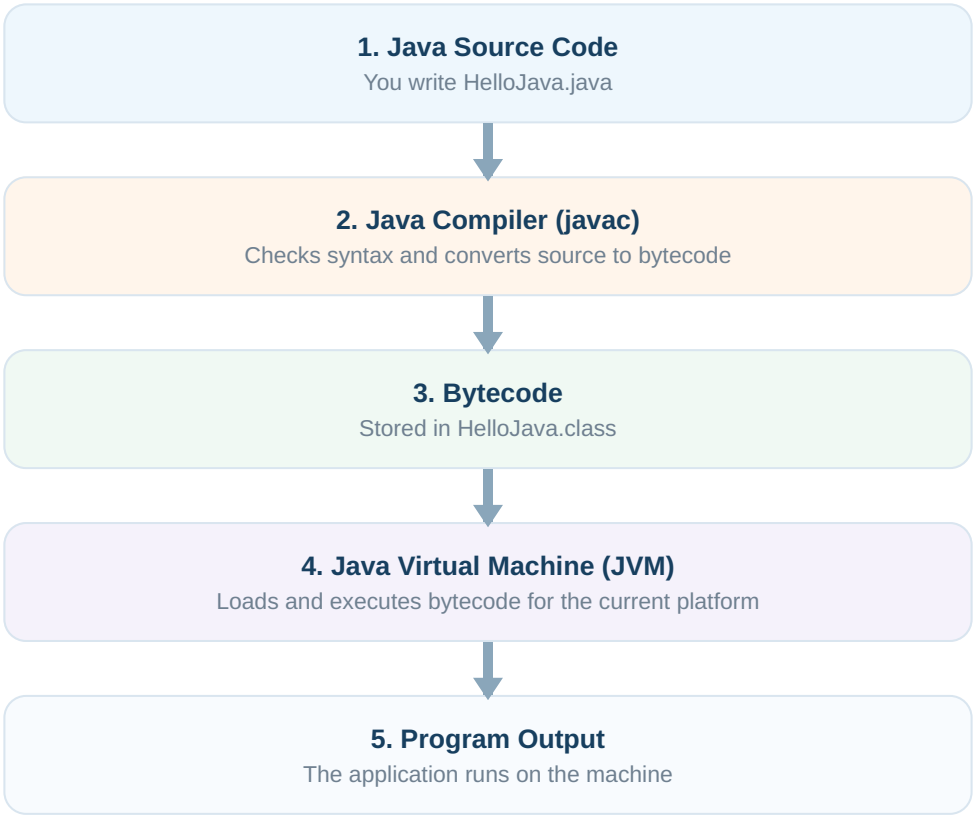
Java SE is not a "beginner version" of Java. It is the core Java platform. Even professional Java developers use Java SE APIs and language features.

Predict: Which edition should a student use to first learn variables, loops, arrays, methods and object-oriented programming?

MODULE 1 - ARCHITECTURE

Java architecture: the big picture

Java separates **writing and compiling** from **running**. This is the central idea behind Java's portability.



KEY IDEA

You normally compile Java source **once** into bytecode. Different JVM implementations make that bytecode executable on different operating systems and processor environments.

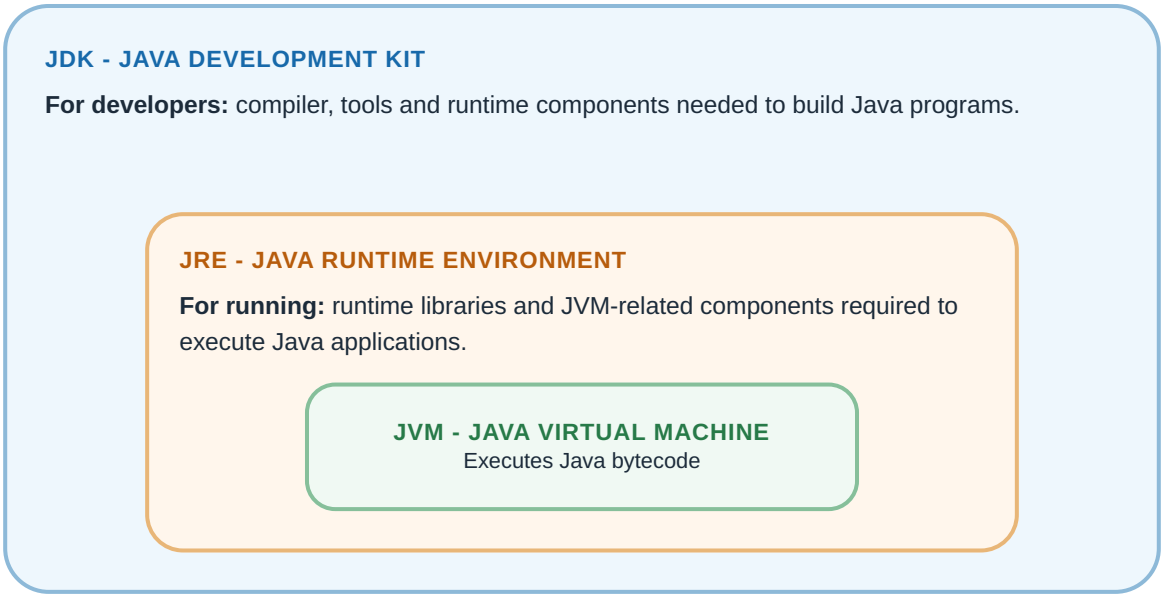
SIMPLIFIED VISUAL

This diagram is intentionally simplified for learning. A real JVM performs class loading, verification, interpretation/JIT compilation, memory management and other runtime work.

MODULE 1 - RUNTIME TOOLS

JDK vs JRE vs JVM

These three terms are related, but they answer different questions: **What do I develop with? What runs Java? What executes bytecode?**



Term	Main purpose	Beginner memory trick
JDK	Develop and compile Java programs.	**D = Development**
JRE	Provides the environment needed to run Java applications.	**R = Runtime**
JVM	Loads and executes Java bytecode.	**V = Virtual Machine**

MODERN JAVA NOTE

Modern JDK distributions commonly provide the tools and runtime together. You may not install a separate standalone JRE in the way older tutorials describe, but the conceptual JDK/JRE/JVM distinction is still useful for learning.

MODULE 1 - COMPILATION

Bytecode and the Java compilation process

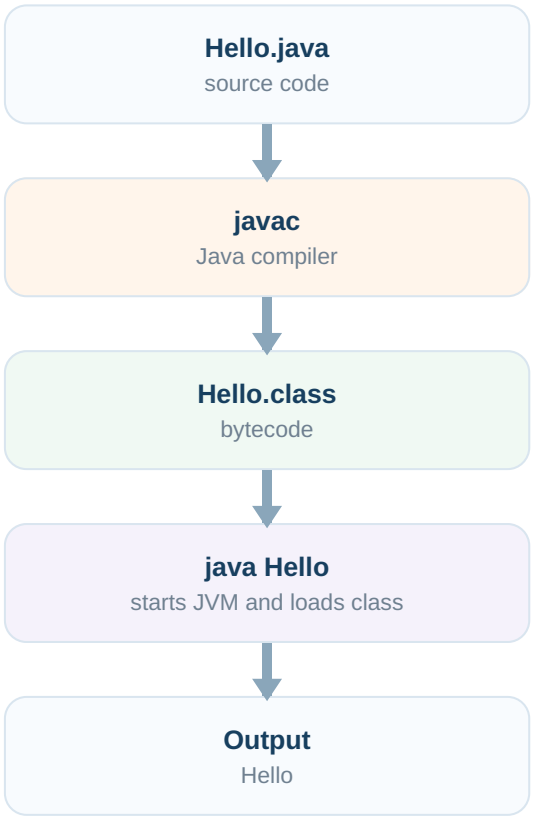
What is bytecode?

Bytecode is the intermediate instruction format produced when Java source code is compiled. It is stored in a **.class** file and is designed to be executed by a JVM.

```
// Source file: Hello.java
public class Hello {
    public static void main(String[]
args) {
        System.out.println("Hello");
    }
}
```

```
javac Hello.java
java Hello
Hello
```

COMPILATION FLOW



COMMON MISTAKE

Do not run **java Hello.java** when you are practicing the classic two-step compile/run process. Compile with **javac Hello.java**, then run the class with **java Hello**.

QUICK RECAP

.java = source code
.class = bytecode
javac = compiler
java = launches the runtime to execute the class

MODULE 1 - SETUP

Installing Java

For learning and development, install a **JDK**. Then verify that both the Java runtime command and compiler command are available.

1 Install a JDK

Use a supported JDK distribution appropriate for your operating system.

2 Open a terminal

On Windows use Command Prompt/ PowerShell; on macOS or Linux use Terminal.

3 Verify the runtime

```
java -version
```

4 Verify the compiler

```
javac -version
```

WHAT SUCCESS LOOKS LIKE

The terminal prints installed Java/Javac version information instead of saying the command is unknown or not found.

IF JAVAC IS MISSING

You may have an incorrect PATH configuration or a runtime-only installation. Make sure the JDK's **bin** directory is available to your terminal environment.

ENVIRONMENT VARIABLE IDEA

PATH tells your operating system where to find commands such as **java** and **javac**.

Mini checkpoint

1. Which tool do you need for compiling Java source code?

2. Which two terminal commands can you run to confirm that Java is installed correctly?

MODULE 1 - FIRST CODE

Your first Java program

```
public class HelloJava {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

```
Hello, Java!
```

Structure of the program

1 Class declaration

public class HelloJava defines a class named HelloJava.

2 Main method

public static void main(String[] args) is the standard entry point used when the JVM starts this class as an application.

3 Statement

System.out.println(...); prints a line of text to standard output.

4 Braces

{ } group the class body and method body.

FILE NAME RULE

If a top-level class is declared **public**, the source filename must match that public class name: **HelloJava.java**.

COMMON MISTAKE

Java is case-sensitive. **String** and **string** are different identifiers. So are **HelloJava** and **hellojava**.

TRY IT YOURSELF

Change the message to your name, compile the file again, and run it. Then add a second **println** statement.

MODULE 1 - REVIEW

Quick recap + practice

ONE-PAGE MEMORY MAP

Programming
turn problems into instructions

↓

Java
high-level, class-based language

↓

javac
source -> bytecode

↓

JVM
bytecode -> execution

JDK = develop

JRE = runtime environment

JVM = execute bytecode

Concept questions

1. Why is Java commonly described as platform independent?

2. What is the difference between a **.java** file and a **.class** file?

3. What is the purpose of **javac**?

4. Which Java edition forms the core foundation for learning the language?

5. Why must **HelloJava.java** match the public class name **HelloJava**?

Find the error

```
public class Welcome {
    public static void Main(String[] args) {
        System.out.println("Welcome")
    }
}
```

YOUR TASK

Identify the issues before checking with an instructor or IDE. Hint: look at the method name and statement ending.

MODULE 2

Java Fundamentals

Now you know how Java runs. This module focuses on the building blocks used inside almost every Java program.

AFTER THIS MODULE, YOU SHOULD BE ABLE TO

- Recognize tokens, keywords and valid identifiers.
- Declare variables using suitable data types and literals.
- Use common operators to build expressions.
- Explain widening and narrowing type casting.
- Write comments, take user input with Scanner and print formatted output.

PROGRAM INGREDIENTS

Names + values + operations =

```
int marks = 85;
int bonus = 5;
int total = marks + bonus;

System.out.println(total);
```

MODULE 2 - LANGUAGE BUILDING BLOCKS

Tokens, keywords and identifiers

A Java program can be broken into small meaningful units called **tokens**. These include keywords, identifiers, literals, operators and separators.

```
int marks = 90;
```

`int` `marks` `=` `90` `;`

KEYWORD

A reserved word with a predefined meaning in Java. Examples include **class**, **public**, **int**, **if** and **return**.

You cannot use a Java keyword as your own variable or class name.

IDENTIFIER

A name created by the programmer for classes, methods, variables and other program elements.

Examples: **studentName**, **calculateTotal**, **BankAccount**.

Identifier rules

- Can use letters, digits, underscore (_) and dollar sign ($).
- Cannot begin with a digit.
- Cannot be a reserved keyword.
- Identifiers are case-sensitive.

GOOD NAMING HABIT

Use meaningful names such as **totalMarks** instead of vague names such as **x**. Java convention commonly uses **camelCase** for variables and methods and **PascalCase** for class names.

Quick Check: Which are valid identifiers: **studentAge**, **2marks**, **class**, **_count**?

MODULE 2 - DATA

Variables and data types

ANALOGY

A variable is a labeled storage box

The label is the variable name. The data type controls what kind of value can be stored. The current value is what is inside the box.


```
int age = 19;
double cgpa = 8.25;
char grade = 'A';
boolean isActive = true;
```

Primitive data types

Type	Used for	Example
byte	small whole numbers	100
short	whole numbers	30000
int	common whole numbers	42
long	large whole numbers	9000000000L
float	decimal values	3.5F
double	common decimal values	3.14159
char	single UTF-16 code unit	'A'
boolean	true/ false condition	true

DECLARATION PATTERN

```
dataType variableName = initialValue;
```

MODULE 2 - DATA

Literals: values written directly in code

A literal is a value written directly in the source code. The literal's form tells Java what kind of value it represents.

INTEGER LITERALS

```
int count = 25;
long population = 1400000000L;
```

BOOLEAN LITERALS

```
boolean passed = true;
boolean absent = false;
```

FLOATING-POINT LITERALS

```
double price = 99.50;
float temperature = 36.5F;
```

CHARACTER AND STRING LITERALS

```
char section = 'A';
String name = "Arun";
```

Character uses single quotes. String uses double quotes.

COMMON MISTAKE

These are not equivalent:

```
char grade = 'A';
String gradeText = "A";
```

One stores a character value; the other refers to a String object.

REMEMBER SUFFIXES

L/l can mark a long literal; **F/f** marks a float literal. Uppercase **L** is clearer because lowercase l can look like the digit 1.

Predict the type

> What kind of literal is each value: **120**, **4.5**, **'J'**, **"Java"**, **false**?

MODULE 2 - OPERATIONS

Operators

Operators perform actions on values or variables. The values they work on are called operands.

```
int total = marks + bonus;
```

`marks` `+` `bonus`

Arithmetic

+	Addition	10 + 5 -> 15
-	Subtraction	10 - 5 -> 5
*****	Multiplication	10 * 5 -> 50
/	Division	10 / 5 -> 2
%	Remainder	10 % 3 -> 1

Relational

==, !=, >, <, >=, <= compare values and produce a boolean result.

Logical

&& AND - both conditions must be true
|| OR - at least one condition must be true
! NOT - reverses a boolean value

```
boolean eligible = age >= 18 && hasId;
```

Assignment

= assigns a value. Compound forms such as **+=**, **-=**, ***=** and **/=** update a variable using an operation.

```
score += 5;  // same idea as score = score + 5
```

COMMON MISTAKE

= means assignment. **==** means equality comparison for primitive values. Mixing them up is a classic beginner error.

MODULE 2 - EXPRESSIONS & CONVERSION

Expressions and type casting

What is an expression?

An expression combines values, variables, operators and method calls to produce a result.

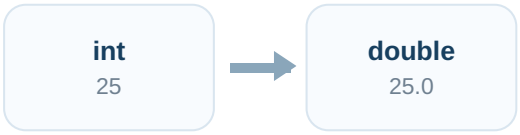
```
int total = mark1 + mark2 + mark3;
double average = total / 3.0;
```

HOW IT WORKS

mark1 + mark2 + mark3 is evaluated first. The result is assigned to **total**. Then **total / 3.0** produces a decimal result because 3.0 is a double literal.

Type casting

Type casting converts a value from one data type to another when the conversion is allowed.



WIDENING

A smaller numeric type converts to a wider compatible type automatically in many cases.

```
int n = 25;
double d = n;
```

NARROWING

A wider numeric type is explicitly cast to a narrower type. Information can be lost.

```
double price = 99.95;
int roundedDown = (int) price;
```

IMPORTANT CONSEQUENCE

Casting **99.95** to **int** produces **99**. It does not round to the nearest integer; the fractional part is discarded.

Predict the output: What does **(int) 7.9** produce? What about assigning **int x = 7; double y = x;**?

MODULE 2 - READABLE CODE

Comments

Comments are notes in source code that help humans understand the program. The compiler ignores them as program instructions.

SINGLE-LINE

```
// Calculate the final mark
int total = theory + practical;
```

MULTI-LINE

```
/*
  This program calculates
  a student's average mark.
*/
```

DOCUMENTATION COMMENT

```
/**
 * Returns the total of two marks.
 */
int addMarks(int a, int b) {
    return a + b;
}
```

Documentation comments can be used by Javadoc tools to generate API documentation.

Good comments explain *why*

WEAK COMMENT

```
// Add 5
score = score + 5;
```

MORE USEFUL

```
// Add attendance bonus
score = score + 5;
```

REMEMBER THIS

Readable names reduce the need for comments. A comment should add context, not repeat code that is already obvious.

MODULE 2 - INTERACTION

User input and output

Reading input with Scanner

```
import java.util.Scanner;

public class StudentInput {
    public static void main(String[] args) {
        Scanner input = new
Scanner(System.in);

        System.out.print("Enter age: ");
        int age = input.nextInt();

        System.out.println("Age: " + age);
        input.close();
    }
}
```

FLOW

Keyboard -> System.in -> Scanner -> Java variable

Common Scanner methods

nextInt()	Reads an int
nextDouble()	Reads a double
next()	Reads one token/word
nextLine()	Reads a full line

Output methods

```
System.out.print("Hello ");
System.out.println("Java");
System.out.printf("Score: %d%n", 95);
```

```
Hello Java
Score: 95
```

COMMON SCANNER ISSUE

After methods such as **nextInt()**, a pending newline can affect a following **nextLine()**. Beginners should learn this behavior when mixing token-based reads with full-line reads.

MODULE 2 - REVIEW

Quick recap + practice

MEMORY MAP

- Token = smallest meaningful unit
- Keyword = reserved word
- Identifier = programmer-defined name
- Variable = named storage
- Literal = value written in code
- Operator = performs an operation
- Expression = produces a result
- Casting = converts type

Predict the output

```
int a = 10;
int b = 3;
System.out.println(a / b);
System.out.println(a % b);
```

THINK FIRST

Both operands in **a / b** are integers. What result do you expect from integer division?

Practice

1. Concept: Explain the difference between a keyword and an identifier.

2. Naming: Write three valid variable names for student marks.

3. Data type: Choose suitable types for age, CGPA, section letter and attendance status.

4. Find the error: Why is **int 2marks = 90;** invalid?

5. Casting: What value is stored after **int x = (int) 12.99;**?

6. Mini challenge: Read a student's name and three marks, calculate the total, and print a clear result.

YOU ARE READY FOR THE NEXT LAYER

If you can explain these fundamentals and write the mini challenge without copying, you have the base needed for control statements, loops, methods and arrays.